

UNIVERSIDAD LA SALLE



Facultad de Ingenierías y Arquitectura
Escuela Profesional de Ingeniería de Software

Curso: Calidad de Software
Docente: Maribel Molina Barriga

PRÁCTICA DE LABORATORIO 02

Diagrama UML y código relevante

Proyecto OLA — Observatorio Litoral de
Anomalías térmicas

*Diagramas de clases del diseño actual y del propuesto, y el código del
módulo «Alertas y avisos» antes y después de la refactorización*

Integrantes

Frederick Mares Graos
Jhordan Huamani Huamani
Jorge Ortiz Castañeda
Miguel Angel Flores Leon
Piero Adrian Delgado Chipana

Arequipa, octubre de 2026

Índice

1. Diagramas de clases	2
2. Código relevante	7
2.1. Antes: un servicio con cinco responsabilidades	7
2.2. Después: canales, despachador y coordinador	11

1. Diagramas de clases

Los diagramas modelan el módulo «Alertas y avisos» del backend de OLA en el commit `d5c054f`, antes y después de la refactorización de `refactorizacion_avisos.patch`. Cada diseño se muestra en dos figuras: el módulo con sus clases y dependencias (a) y las clases de `db.models` (b). Se dibujaron con el sistema visual de Archify; el generador está en `diagramas/fuente/`.

Como el backend agrupa gran parte de su lógica en módulos de funciones, estos se representan como clases con el estereotipo «module», y sus miembros son las funciones y los métodos medidos (NOM).

Cómo leer las figuras.

- En el diseño actual (Figuras 1a y 1b), el rosa marca las unidades y dependencias con hallazgos de la matriz.
- En el diseño propuesto (Figuras 2a y 2b), el verde marca lo nuevo, el ámbar lo modificado y el rosa los hallazgos que siguen pendientes.
- Las figuras 1b y 2b muestran todos los atributos de las cuatro entidades del módulo y las asociaciones que derivan de sus claves foráneas.

Diseño actual del módulo «Alertas y avisos» – diagrama de clases

Commit d5c054f · E-01a · En rosa, las unidades y dependencias con hallazgos (H-xx). Los atributos de las entidades están en la Figura 1b.

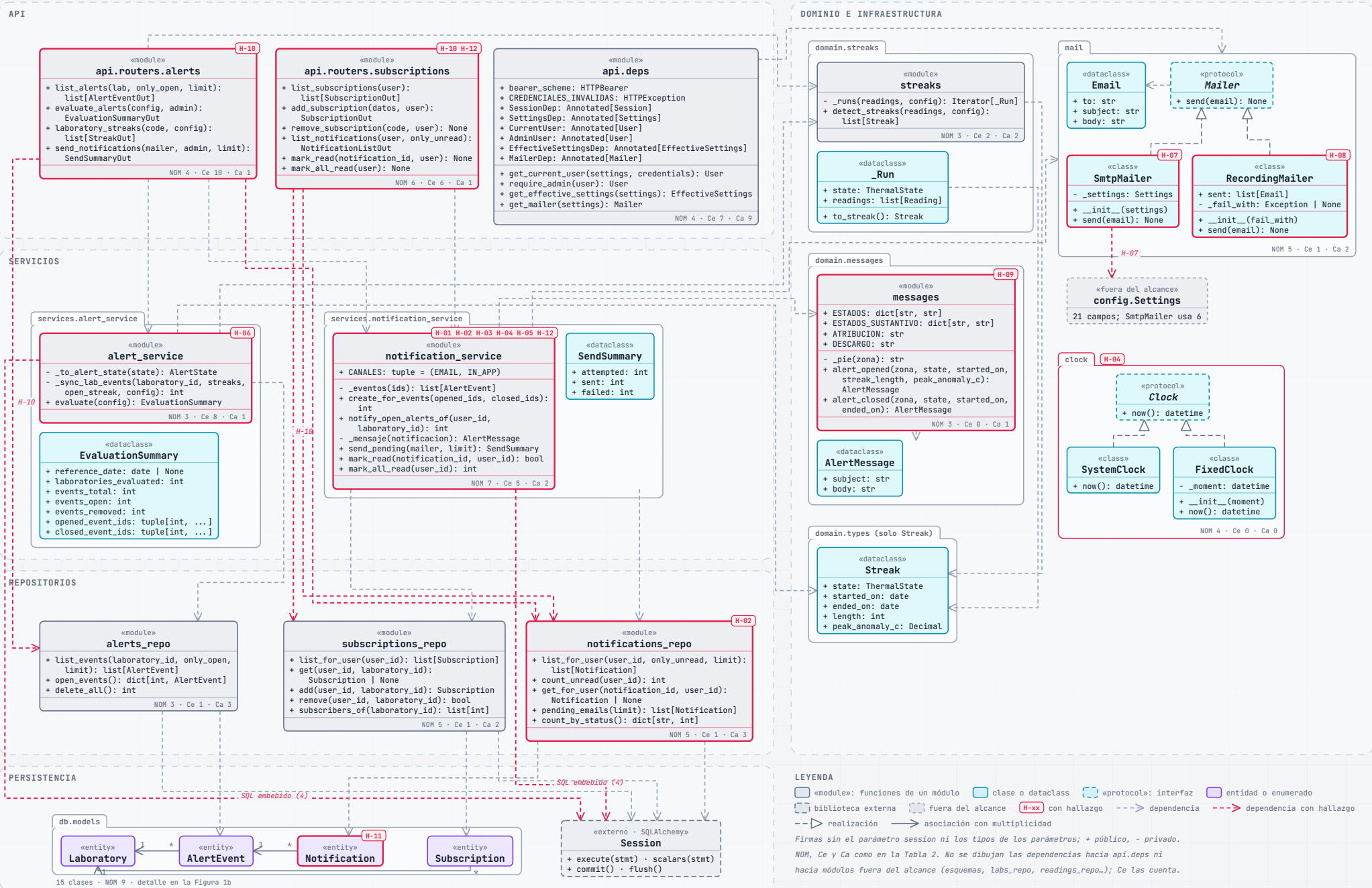
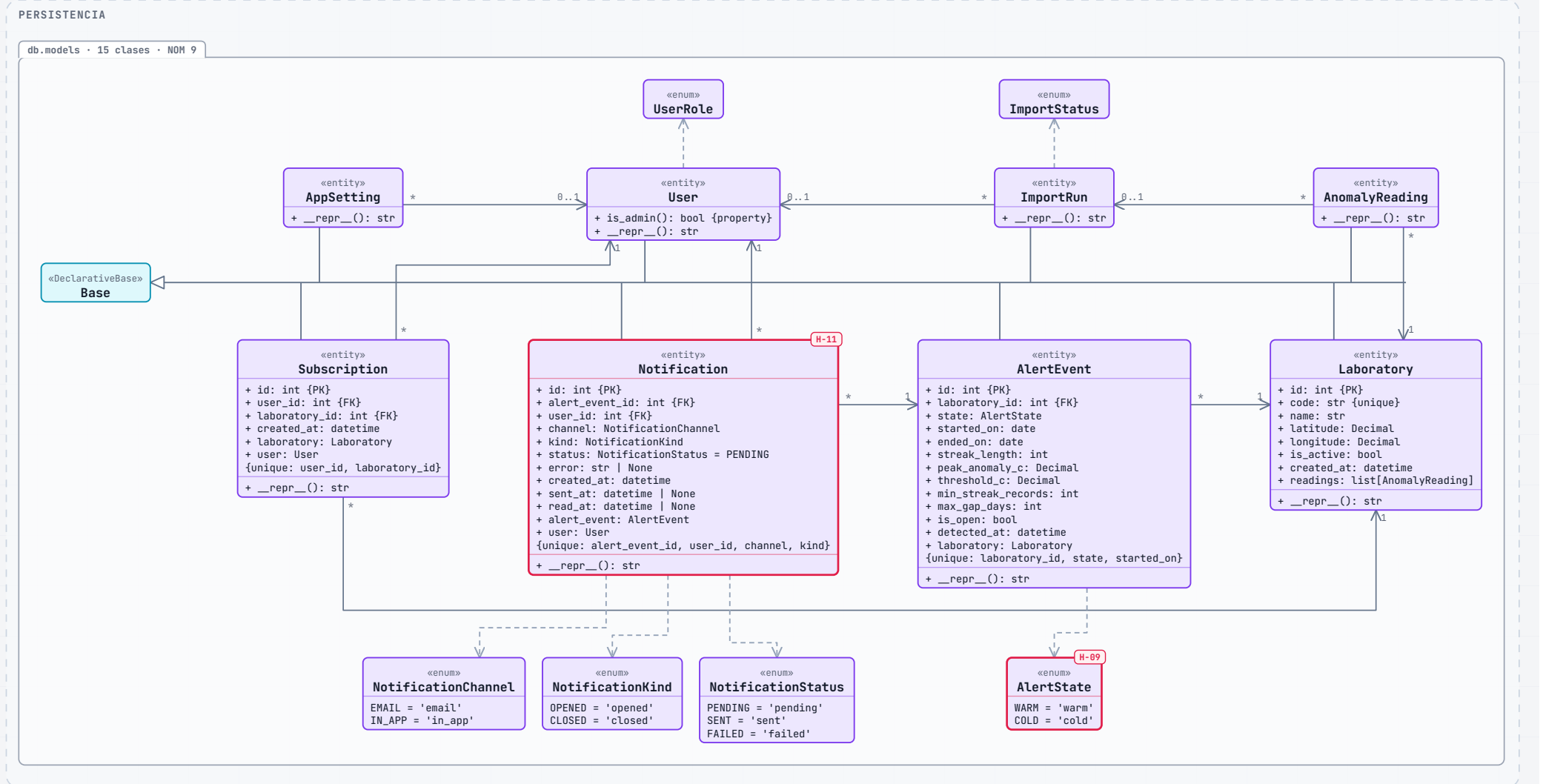


Figura 1a: E-01. Diagrama de clases del diseño actual del módulo «Alertas y avisos». En rosa, las unidades y dependencias con hallazgos.

Diseño actual – las 15 clases de db.models

Commit d5c054f · E-01b · Atributos completos en las clases del módulo «Alertas y avisos»; en las demás, nombre, estereotipo y métodos. NOM 9.



LEYENDA

«module»: funciones de un módulo «class» clase o dataclass «protocol»: interfaz «entity» entidad o enumerado «library» biblioteca externa «out of scope» fuera del alcance «H-xx» con hallazgo «-->» dependencia «-->» dependencia con hallazgo

«-->» realización «-->» generalización «-->» asociación con multiplicidad

Todas las entidades heredan de Base (árbol de generalización). {PK}, {FK} y {unique} vienen de las columnas y restricciones de SQLAlchemy.

Diseño propuesto del módulo «Alertas y avisos» – diagrama de clases

d5c054f • refactorizacion_avisos.patch • E-06a • Verde: nuevo; ámbar: modificado; rosa: hallazgos pendientes (H-xx). Los atributos de las entidades están en la Figura 2b.

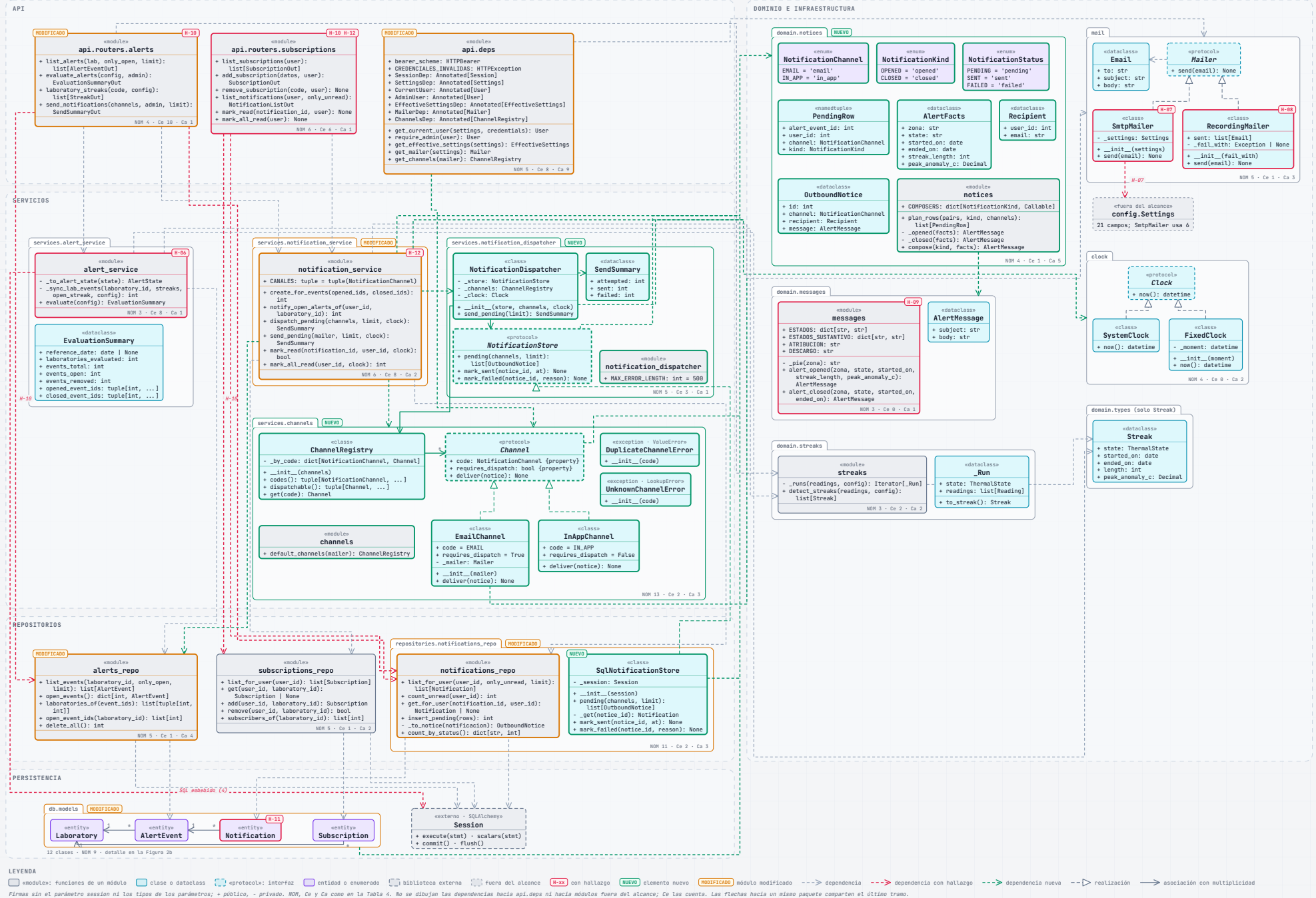


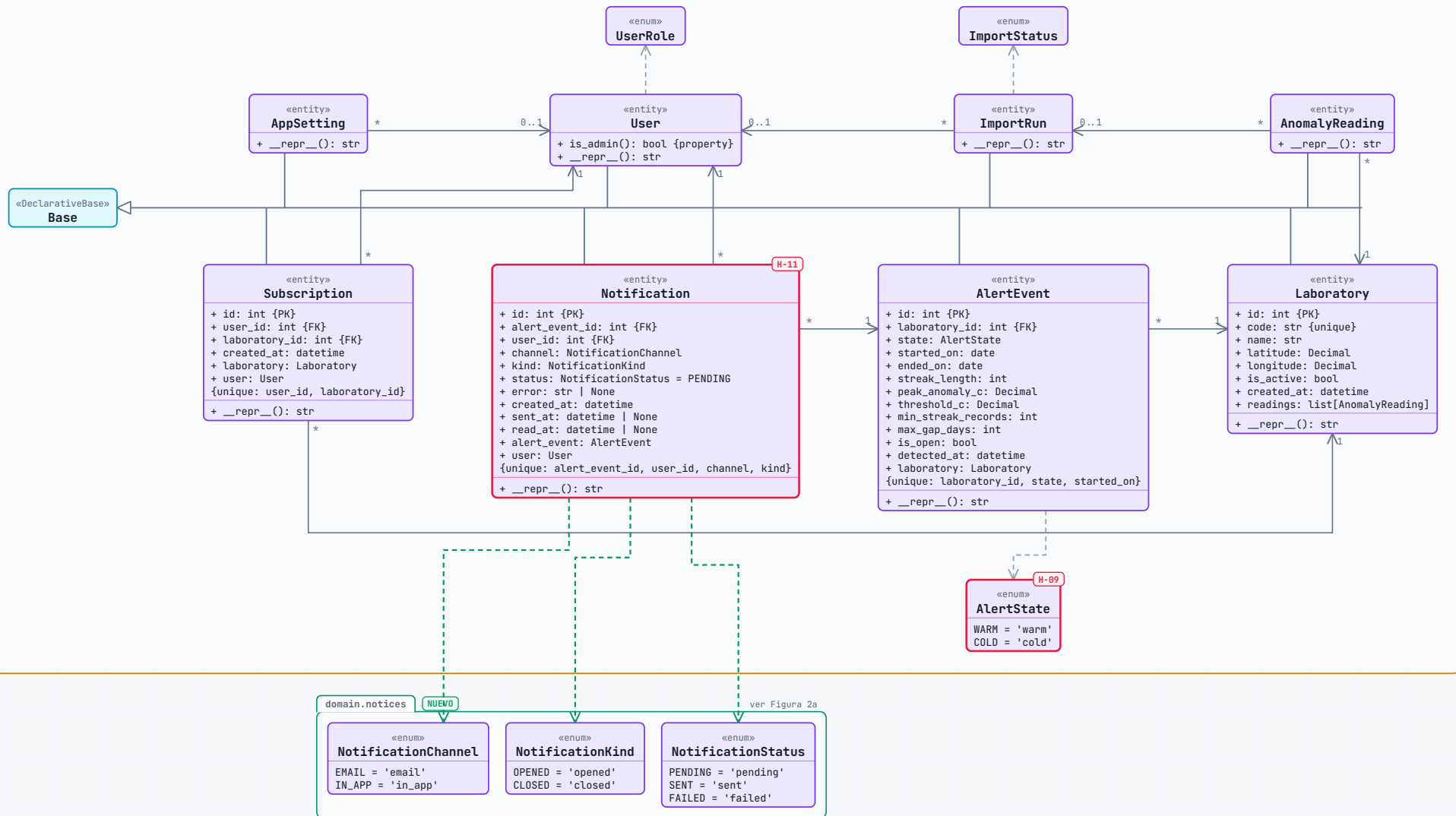
Figura 2a: E-06. Diagrama de clases del diseño propuesto del módulo «Alertas y avisos». Verde: nuevo; ámbar: modificado; rosa: hallazgos pendientes.

Diseño propuesto – las clases de db.models

E-06b · Los enumerados de los avisos pasan a domain.notices y db.models los importa (la persistencia depende del dominio). En rosa, los hallazgos que siguen pendientes.

PERSISTENCIA Y DOMINIO

db.models · 12 clases · NOM 9 **MODIFICADO**



LEYENDA

«module»: funciones de un módulo «protocol»: interfaz «entity» entidad o enumerado «enum» enumerado «library» biblioteca externa «out of scope» fuera del alcance «H-xx» con hallazgo «MODIFICADO» módulo modificado «NUEVO» elemento nuevo
 --> dependencia -.-> dependencia con hallazgo -.-> dependencia nueva --> realización --> generalización --> asociación con multiplicidad
 Todas las entidades heredan de Base (árbol de generalización). {PK}, {FK} y {unique} vienen de las columnas y restricciones de SQLAlchemy.

2. Código relevante

Se incluyen completos los cuatro archivos que definen el rediseño y, como contraste, el servicio original que reunía las responsabilidades. Los archivos tienen su numeración de líneas. El cambio completo, con las pruebas unitarias nuevas, está en `refactorizacion_avisos.patch`.

2.1 Antes: un servicio con cinco responsabilidades

`notification_service` (141 líneas de código, complejidad acumulada 20) decide qué avisos corresponden a cada episodio, construye sentencias SQL, redacta el mensaje, envía el correo y marca la lectura. La lista de canales es la tupla `CANALES` y `send_pending` solo sabe enviar correo (hallazgos H-01 a H-05).

Fragmento 1: E-05. Diseño actual: `services/notification_service.py` en el commit `d5c054f`.

```

1  """Avisos a los usuarios suscritos (RF-03).
2
3  El envio esta separado en dos pasos a proposito:
4
5  1. Al evaluar las alertas se REGISTRAN los avisos pendientes. Es rapido y no
6     depende del servidor de correo.
7  2. Un endpoint de administrador dispara el ENVIO. Asi un servidor lento o
8     caido no bloquea ni hace fallar la evaluacion, y los fallos quedan
9     visibles para reintentar.
10
11 La restriccion de unicidad de la tabla es la que impide repetir un aviso: por
12 mucho que se reevalue, un episodio genera un unico correo por usuario.
13 """
14
15 from __future__ import annotations
16
17 from collections.abc import Sequence
18 from dataclasses import dataclass
19 from datetime import UTC, datetime
20
21 from sqlalchemy import select
22 from sqlalchemy.dialects.postgresql import insert
23 from sqlalchemy.orm import Session, joinedload
24
25 from ola.db.models import (
26     AlertEvent,
27     Notification,
28     NotificationChannel,
29     NotificationKind,
30     NotificationStatus,
31 )
32 from ola.domain.messages import AlertMessage, alert_closed, alert_opened
33 from ola.mail import Email, Mailer
34 from ola.repositories import notifications_repo, subscriptions_repo
35
36 CANALES = (NotificationChannel.EMAIL, NotificationChannel.IN_APP)
37
38 @dataclass(frozen=True)
39 class SendSummary:
40     attempted: int
41     sent: int
42     failed: int
43
44
45 def _eventos(session: Session, ids: Sequence[int]) -> list[AlertEvent]:
46     if not ids:
47         return []
48     return list(

```

```

50     session.scalars(
51         select(AlertEvent)
52         .options(joinedload(AlertEvent.laboratory))
53         .where(AlertEvent.id.in_(ids))
54     )
55 )
56
57
58 def create_for_events(
59     session: Session, *, opened_ids: Sequence[int] = (), closed_ids: Sequence[int] = ()
60 ) -> int:
61     """Registra los avisos pendientes de los episodios indicados.
62
63     Devuelve cuantos se crearon realmente: los que ya existian se ignoran, de
64     modo que llamar dos veces no duplica nada.
65     """
66     filas: list[dict[str, object]] = []
67
68     for tipo, ids in ((NotificationKind.OPENED, opened_ids), (NotificationKind.CLOSED, closed_ids)):
69         for evento in _eventos(session, ids):
70             for user_id in subscriptions_repo.subscribers_of(session, evento.laboratory_id):
71                 filas.extend(
72                     {
73                         "alert_event_id": evento.id,
74                         "user_id": user_id,
75                         "channel": canal,
76                         "kind": tipo,
77                         "status": NotificationStatus.PENDING,
78                     }
79                     for canal in CANALES
80                 )
81
82     if not filas:
83         return 0
84
85     creados = session.execute(
86         insert(Notification)
87         .values(filas)
88         .on_conflict_do_nothing(constraint="uq_notification_unica")
89         .returning(Notification.id)
90     ).scalars()
91     total = len(list(creados))
92     session.commit()
93     return total
94
95
96 def notify_open_alerts_of(session: Session, user_id: int, laboratory_id: int) -> int:
97     """Avisa de la alerta vigente de una zona a quien acaba de suscribirse.
98
99     Sin esto, quien se suscribe a una zona que YA esta en alerta no se enterara
100     hasta que el episodio termine, que no es lo que espera: en CALLAO hay uno
101     abierto desde abril. La restriccion de unicidad evita duplicar el aviso si
102     la persona se da de baja y se vuelve a suscribir.
103     """
104     vigentes = list(
105         session.scalars(
106             select(AlertEvent).where(
107                 AlertEvent.laboratory_id == laboratory_id, AlertEvent.is_open.is_(True)
108             )
109         )
110     )
111     if not vigentes:
112         return 0
113
114     filas = [
115         {
116             "alert_event_id": evento.id,
117             "user_id": user_id,
118             "channel": canal,

```

```

119         "kind": NotificationKind.OPENED,
120         "status": NotificationStatus.PENDING,
121     }
122     for evento in vigentes
123     for canal in CANALES
124 ]
125 creados = session.execute(
126     insert(Notification)
127     .values(filas)
128     .on_conflict_do_nothing(constraint="uq_notification_unica")
129     .returning(Notification.id)
130 ).scalars()
131 return len(list(creados))
132
133
134 def _mensaje(notificacion: Notification) -> AlertMessage:
135     evento = notificacion.alert_event
136     zona = evento.laboratory.name
137     if notificacion.kind is NotificationKind.OPENED:
138         return alert_opened(
139             zona=zona,
140             state=evento.state.value,
141             started_on=evento.started_on,
142             streak_length=evento.streak_length,
143             peak_anomaly_c=evento.peak_anomaly_c,
144         )
145     return alert_closed(
146         zona=zona,
147         state=evento.state.value,
148         started_on=evento.started_on,
149         ended_on=evento.ended_on,
150     )
151
152
153 def send_pending(session: Session, mailer: Mailer, *, limit: int = 200) -> SendSummary:
154     """Envia los correos pendientes y los que fallaron en un intento anterior.
155
156     Un fallo no detiene la tanda ni pierde el aviso: queda marcado con su
157     motivo para que el administrador pueda reintentar.
158     """
159     pendientes = notifications_repo.pending_emails(session, limit=limit)
160     enviados = fallidos = 0
161
162     for notificacion in pendientes:
163         mensaje = _mensaje(notificacion)
164         try:
165             mailer.send(
166                 Email(to=notificacion.user.email, subject=mensaje.subject, body=mensaje.body)
167             )
168         except Exception as exc:
169             notificacion.status = NotificationStatus.FAILED
170             notificacion.error = str(exc)[:500]
171             fallidos += 1
172         else:
173             notificacion.status = NotificationStatus.SENT
174             notificacion.error = None
175             notificacion.sent_at = datetime.now(UTC)
176             enviados += 1
177
178     session.commit()
179     return SendSummary(attempted=len(pendientes), sent=enviados, failed=fallidos)
180
181
182 def mark_read(session: Session, notification_id: int, user_id: int) -> bool:
183     notificacion = notifications_repo.get_for_user(session, notification_id, user_id)
184     if notificacion is None:
185         return False
186     if notificacion.read_at is None:
187         notificacion.read_at = datetime.now(UTC)

```

```
188         session.commit()
189     return True
190
191
192 def mark_all_read(session: Session, user_id: int) -> int:
193     pendientes = notifications_repo.list_for_user(session, user_id, only_unread=True, limit=500)
194     ahora = datetime.now(UTC)
195     for notificacion in pendientes:
196         notificacion.read_at = ahora
197     session.commit()
198     return len(pendientes)
```

2.2 Después: canales, despachador y coordinador

Canales. Cada canal cumple el contrato `Channel`: declara si requiere envío y sabe entregar un aviso. `ChannelRegistry` los indexa y `default_channels` es el único lugar donde se decide cuáles tiene el sistema.

Fragmento 2: E-07. Diseño propuesto: `services/channels.py` (contrato, canales y registro).

```

1  """Canales de entrega de los avisos (RF-03).
2
3  Cada canal sabe entregar un aviso y nada mas. Quien envia (el despachador)
4  solo conoce el contrato 'Channel', asi que un canal nuevo se anade
5  escribiendo una clase y registrandola en 'default_channels', sin modificar
6  el envio, el repositorio ni los routers.
7  """
8
9  from __future__ import annotations
10
11 from collections.abc import Iterable
12 from typing import Protocol
13
14 from ola.domain.notices import NotificationChannel, OutboundNotice
15 from ola.mail import Email, Mailer
16
17
18 class Channel(Protocol):
19     @property
20     def code(self) -> NotificationChannel: ...
21
22     @property
23     def requires_dispatch(self) -> bool:
24         """Falso en los canales de consulta, donde el aviso guardado ya es la entrega."""
25         ...
26
27     def deliver(self, notice: OutboundNotice) -> None:
28         """Entrega el aviso. Lanza una excepcion si falla; quien llama la registra."""
29         ...
30
31
32 class EmailChannel:
33     code = NotificationChannel.EMAIL
34     requires_dispatch = True
35
36     def __init__(self, mailer: Mailer) -> None:
37         self._mailer = mailer
38
39     def deliver(self, notice: OutboundNotice) -> None:
40         self._mailer.send(
41             Email(
42                 to=notice.recipient.email,
43                 subject=notice.message.subject,
44                 body=notice.message.body,
45             )
46         )
47
48
49 class InAppChannel:
50     """El aviso se lee desde la aplicacion: no hay nada que empujar."""
51
52     code = NotificationChannel.IN_APP
53     requires_dispatch = False
54
55     def deliver(self, notice: OutboundNotice) -> None:
56         return None
57
58
59 class DuplicateChannelError(ValueError):
60     def __init__(self, code: NotificationChannel) -> None:

```

```
61     super().__init__(f"El canal '{code.value}' esta registrado mas de una vez.")
62
63
64 class UnknownChannelError(LookupError):
65     def __init__(self, code: NotificationChannel) -> None:
66         super().__init__(f"No hay ningun canal registrado para '{code.value}'.")
67
68
69 class ChannelRegistry:
70     """Los canales disponibles, indexados por su codigo."""
71
72     def __init__(self, channels: Iterable[Channel]) -> None:
73         self._by_code: dict[NotificationChannel, Channel] = {}
74         for channel in channels:
75             if channel.code in self._by_code:
76                 raise DuplicateChannelError(channel.code)
77             self._by_code[channel.code] = channel
78
79     def codes(self) -> tuple[NotificationChannel, ...]:
80         return tuple(self._by_code)
81
82     def dispatchable(self) -> tuple[Channel, ...]:
83         return tuple(c for c in self._by_code.values() if c.requires_dispatch)
84
85     def get(self, code: NotificationChannel) -> Channel:
86         try:
87             return self._by_code[code]
88         except KeyError:
89             raise UnknownChannelError(code) from None
90
91
92 def default_channels(mailer: Mailer) -> ChannelRegistry:
93     """Punto unico donde se decide que canales tiene el sistema."""
94     return ChannelRegistry([EmailChannel(mailer), InAppChannel()])
```

Despachador. NotificationDispatcher contiene la lógica de envío (reintentos, registro del fallo y límite de la tanda). Recibe por el constructor el almacén, los canales y el reloj, definidos como protocolos, y por eso se prueba sin base de datos.

Fragmento 3: E-07. Diseño propuesto: services/notification_dispatcher.py.

```

1  """Envío de los avisos pendientes (RF-03).
2
3  El despachador decide QUE se envia y que pasa cuando falla. No sabe de
4  SQLAlchemy, de SMTP ni del reloj del sistema: recibe un almacen, los canales
5  y un reloj. Por eso se prueba con dobles en memoria, sin base de datos.
6  """
7
8  from __future__ import annotations
9
10 from collections.abc import Sequence
11 from dataclasses import dataclass
12 from datetime import datetime
13 from typing import Protocol
14
15 from ola.clock import Clock
16 from ola.domain.notices import NotificationChannel, OutboundNotice
17 from ola.services.channels import ChannelRegistry
18
19 # La columna 'error' es de texto libre, pero un volcado de pila no ayuda a
20 # quien revisa los fallos desde la aplicacion.
21 MAX_ERROR_LENGTH = 500
22
23
24 class NotificationStore(Protocol):
25     def pending(
26         self, channels: Sequence[NotificationChannel], *, limit: int
27     ) -> list[OutboundNotice]:
28         """Avisos por entregar de esos canales, incluidos los que fallaron antes."""
29         ...
30
31     def mark_sent(self, notice_id: int, at: datetime) -> None: ...
32
33     def mark_failed(self, notice_id: int, reason: str) -> None: ...
34
35
36 @dataclass(frozen=True)
37 class SendSummary:
38     attempted: int
39     sent: int
40     failed: int
41
42
43 class NotificationDispatcher:
44     def __init__(self, store: NotificationStore, channels: ChannelRegistry, clock: Clock) -> None:
45         self._store = store
46         self._channels = channels
47         self._clock = clock
48
49     def send_pending(self, *, limit: int = 200) -> SendSummary:
50         """Entrega los avisos pendientes y los que fallaron en un intento anterior.
51
52         Un fallo no detiene la tanda ni pierde el aviso: queda marcado con su
53         motivo para que el administrador pueda reintentar.
54         """
55         codigos = [canal.code for canal in self._channels.dispatchable()]
56         pendientes = self._store.pending(codigos, limit=limit)
57         enviados = fallidos = 0
58
59         for aviso in pendientes:
60             try:
61                 self._channels.get(aviso.channel).deliver(aviso)
62             except Exception as exc:
63                 self._store.mark_failed(aviso.id, str(exc)[:MAX_ERROR_LENGTH])

```

```

64         fallidos += 1
65     else:
66         self._store.mark_sent(aviso.id, self._clock.now())
67         enviados += 1
68
69     return SendSummary(attempted=len(pendientes), sent=enviados, failed=fallidos)

```

Dominio de los avisos. `domain/notices.py` reúne los enumerados de canal, tipo y estado (antes en `db/models.py`), la planificación de las filas y la redacción de cada tipo de aviso, sin depender de la infraestructura.

Fragmento 4: E-07. Diseño propuesto: `domain/notices.py`.

```

1  """Conceptos de los avisos a usuarios (RF-03).
2
3  Sin dependencias de base de datos, de FastAPI ni de correo: aqui vive lo que
4  un aviso ES. Como se guarda lo decide 'repositories', y como se entrega,
5  'services.channels'. La capa de persistencia importa estos tipos, y no al
6  reves, de modo que la logica de avisos se prueba sin levantar PostgreSQL.
7  """
8
9  from __future__ import annotations
10
11  import enum
12  from collections.abc import Callable, Iterable, Sequence
13  from dataclasses import dataclass
14  from datetime import date
15  from decimal import Decimal
16  from typing import NamedTuple
17
18  from ola.domain.messages import AlertMessage, alert_closed, alert_opened
19
20
21  class NotificationChannel(enum.StrEnum):
22      EMAIL = "email"
23      IN_APP = "in_app"
24
25
26  class NotificationKind(enum.StrEnum):
27      """Momento del episodio que motiva el aviso."""
28
29      OPENED = "opened"
30      CLOSED = "closed"
31
32
33  class NotificationStatus(enum.StrEnum):
34      PENDING = "pending"
35      SENT = "sent"
36      FAILED = "failed"
37
38
39  class PendingRow(NamedTuple):
40      """Un aviso por registrar: episodio, destinatario, canal y momento."""
41
42      alert_event_id: int
43      user_id: int
44      channel: NotificationChannel
45      kind: NotificationKind
46
47
48  def plan_rows(
49      pairs: Iterable[tuple[int, int]],
50      kind: NotificationKind,
51      channels: Sequence[NotificationChannel],
52  ) -> list[PendingRow]:
53      """Un aviso por cada par (episodio, usuario) y por cada canal."""
54      return [
55          PendingRow(alert_event_id=event_id, user_id=user_id, channel=channel, kind=kind)

```

```

56         for event_id, user_id in pairs
57             for channel in channels
58     ]
59
60
61 @dataclass(frozen=True)
62 class AlertFacts:
63     """Datos del episodio que necesita el texto de un aviso."""
64
65     zona: str
66     state: str
67     started_on: date
68     ended_on: date
69     streak_length: int
70     peak_anomaly_c: Decimal
71
72
73 def _opened(facts: AlertFacts) -> AlertMessage:
74     return alert_opened(
75         zona=facts.zona,
76         state=facts.state,
77         started_on=facts.started_on,
78         streak_length=facts.streak_length,
79         peak_anomaly_c=facts.peak_anomaly_c,
80     )
81
82
83 def _closed(facts: AlertFacts) -> AlertMessage:
84     return alert_closed(
85         zona=facts.zona,
86         state=facts.state,
87         started_on=facts.started_on,
88         ended_on=facts.ended_on,
89     )
90
91
92 # Un tipo de aviso nuevo se anade aqui, sin tocar a quien envia.
93 COMPOSERS: dict[NotificationKind, Callable[[AlertFacts], AlertMessage]] = {
94     NotificationKind.OPENED: _opened,
95     NotificationKind.CLOSED: _closed,
96 }
97
98
99 def compose(kind: NotificationKind, facts: AlertFacts) -> AlertMessage:
100     """Redacta el aviso que corresponde a ese momento del episodio."""
101     return COMPOSERS[kind](facts)
102
103
104 @dataclass(frozen=True)
105 class Recipient:
106     user_id: int
107     email: str
108
109
110 @dataclass(frozen=True)
111 class OutboundNotice:
112     """Un aviso listo para entregar, ya separado de como esta guardado."""
113
114     id: int
115     channel: NotificationChannel
116     recipient: Recipient
117     message: AlertMessage

```

Coordinador. `notification_service` queda con 73 líneas y complejidad acumulada 12: compone el despachador, fija el límite de la transacción y conserva sus firmas públicas, de modo que las pruebas de integración existentes no cambian.

Fragmento 5: E-07. Diseño propuesto: `services/notification_service.py` como coordinador.

```

1  """Avisos a los usuarios suscritos (RF-03).
2
3  El envio esta separado en dos pasos a proposito:
4
5  1. Al evaluar las alertas se REGISTRAN los avisos pendientes. Es rapido y no
6     depende del servidor de correo.
7  2. Un endpoint de administrador dispara el ENVIO. Asi un servidor lento o
8     caido no bloquea ni hace fallar la evaluacion, y los fallos quedan
9     visibles para reintentar.
10
11  Este modulo solo coordina y fija el limite de cada transaccion. Que avisos
12  corresponden lo decide 'domain.notices'; como se guardan, 'repositories';
13  como se entregan, 'services.channels'; y que pasa cuando una entrega falla,
14  'services.notification_dispatcher'.
15  """
16
17  from __future__ import annotations
18
19  from collections.abc import Sequence
20
21  from sqlalchemy.orm import Session
22
23  from ola.clock import Clock, SystemClock
24  from ola.domain.notices import NotificationChannel, NotificationKind, plan_rows
25  from ola.mail import Mailer
26  from ola.repositories import alerts_repo, notifications_repo, subscriptions_repo
27  from ola.repositories.notifications_repo import SqlNotificationStore
28  from ola.services.channels import ChannelRegistry, default_channels
29  from ola.services.notification_dispatcher import NotificationDispatcher, SendSummary
30
31  __all__ = [
32      "SendSummary",
33      "create_for_events",
34      "dispatch_pending",
35      "mark_all_read",
36      "mark_read",
37      "notify_open_alerts_of",
38      "send_pending",
39  ]
40
41  # Cada episodio se registra en todos los canales que el sistema conoce.
42  CANALES: tuple[NotificationChannel, ...] = tuple(NotificationChannel)
43
44
45  def create_for_events(
46      session: Session, *, opened_ids: Sequence[int] = (), closed_ids: Sequence[int] = ()
47  ) -> int:
48      """Registra los avisos pendientes de los episodios indicados.
49
50      Devuelve cuantos se crearon realmente: los que ya existian se ignoran, de
51      modo que llamar dos veces no duplica nada.
52      """
53      filas = []
54      for tipo, ids in ((NotificationKind.OPENED, opened_ids), (NotificationKind.CLOSED, closed_ids)):
55          for event_id, laboratory_id in alerts_repo.laboratories_of(session, ids):
56              suscritos = subscriptions_repo.subscribers_of(session, laboratory_id)
57              filas += plan_rows(((event_id, user_id) for user_id in suscritos), tipo, CANALES)
58
59      if not filas:
60          return 0
61
62      creados = notifications_repo.insert_pending(session, filas)
63      session.commit()

```

```

64     return creados
65
66
67 def notify_open_alerts_of(session: Session, user_id: int, laboratory_id: int) -> int:
68     """Avisa de la alerta vigente de una zona a quien acaba de suscribirse.
69
70     Sin esto, quien se suscribe a una zona que YA esta en alerta no se entera
71     hasta que el episodio termine, que no es lo que espera: en CALLAO hay uno
72     abierto desde abril. La restriccion de unicidad evita duplicar el aviso si
73     la persona se da de baja y se vuelve a suscribir.
74
75     No confirma la transaccion: lo hace quien registra la suscripcion.
76     """
77     vigentes = alerts_repo.open_event_ids(session, laboratory_id)
78     filas = plan_rows(
79         ((event_id, user_id) for event_id in vigentes), NotificationKind.OPENED, CANALES
80     )
81     return notifications_repo.insert_pending(session, filas)
82
83
84 def dispatch_pending(
85     session: Session,
86     channels: ChannelRegistry,
87     *,
88     limit: int = 200,
89     clock: Clock | None = None,
90 ) -> SendSummary:
91     """Entrega los avisos pendientes por los canales recibidos."""
92     despachador = NotificationDispatcher(
93         SqlNotificationStore(session), channels, clock or SystemClock()
94     )
95     resumen = despachador.send_pending(limit=limit)
96     session.commit()
97     return resumen
98
99
100 def send_pending(
101     session: Session, mailer: Mailer, *, limit: int = 200, clock: Clock | None = None
102 ) -> SendSummary:
103     """Envia los correos pendientes y los que fallaron en un intento anterior."""
104     return dispatch_pending(session, default_channels(mailer), limit=limit, clock=clock)
105
106
107 def mark_read(
108     session: Session, notification_id: int, user_id: int, *, clock: Clock | None = None
109 ) -> bool:
110     notificacion = notifications_repo.get_for_user(session, notification_id, user_id)
111     if notificacion is None:
112         return False
113     if notificacion.read_at is None:
114         notificacion.read_at = (clock or SystemClock()).now()
115         session.commit()
116     return True
117
118
119 def mark_all_read(session: Session, user_id: int, *, clock: Clock | None = None) -> int:
120     pendientes = notifications_repo.list_for_user(session, user_id, only_unread=True, limit=500)
121     ahora = (clock or SystemClock()).now()
122     for notificacion in pendientes:
123         notificacion.read_at = ahora
124     session.commit()
125     return len(pendientes)

```